

UninaTaskBoard

Autore	Docenti
Gruppo OOBD24	Prof.ssa Mara SANGIOVANNI
Maeve Cardoni - DE1000425	Prof. Sergio DI MARTINO

**CORSO DI LAUREA IN INFORMATICA
INSEGNAMENTO DI BASI DI DATI
E PROGRAMMAZIONE OBJECT-ORIENTED
ANNO ACCADEMICO 2026-2027**

SOMMARIO

Descrizione del progetto	4
1.1 Cos'è UninaTaskBoard	4
1.2 Specifica delle funzionalità per Programmazione Object-Oriented	4
1.3 Requisiti	4
Progettazione concettuale	6
2.1 Class Diagram UML non ristrutturato	6
2.2 Class Diagram UML Ristrutturato	7
2.3 Diagramma entità-relazioni (ER)	8
Dizionario	9
3.1 Dizionario delle classi	9
3.2 Dizionario dei vincoli	10
Schema logico	11
4.1 Mapping	11
4.2 Schema logico	11
Progettazione Object-Oriented	12
5.1 Mock-up	12
5.2 Class Diagram EBC	13
Schema fisico	14
6.1 Tabelle	14
6.2 Vincoli	15

Documenti di riferimento

Tracce-Progetti-BD-POO-2026-2027

Revisioni

Nome del documento	Versione	Data	Note
--------------------	----------	------	------

Revisioni			
Documentazione- BasiDiDati- OOBD2526_24.pdf	0.1	4 Maggio	Troppe aggregazioni, incorporazione non richiesta di classi inerenti della consegna di POO, utilizzo non richiesto di metodi, nome delle relazioni mancanti, classe creatore mancante
Documentazione- BasiDiDati- OOBD2526_24.PDF	1.0	3 Luglio	Documentazione completa

Descrizione del progetto

1.1 Cos'è UninaTaskBoard

UninaTaskBoard è una piattaforma che consente agli studenti dell'Università di organizzare e gestire attività collaborative all'interno di progetti, come ad esempio progetti di gruppo, preparazione di esami o sviluppo di piccoli applicativi. Gli utenti possono creare uno o più progetti e invitare altri utenti a partecipare. Ogni progetto è composto da un insieme di membri che collaborano allo svolgimento delle attività. All'interno di un progetto è possibile definire diverse attività, ciascuna caratterizzata da una descrizione, una data di creazione, una eventuale scadenza e uno stato di avanzamento. Ogni attività può essere assegnata a uno o più membri del progetto. Le attività possono essere di due tipi: attività di documentazione e attività di sviluppo. Le attività di sviluppo riguardano la realizzazione o la modifica di codice sorgente. Si utilizzino le proprie conoscenze del dominio per definire eventuali dettagli non specificati nella traccia.

1.2 Specifica delle funzionalità per Programmazione Object-Oriented

Si sviluppi un applicativo Java con interfaccia grafica (Swing o JavaFX) per la gestione delle attività collaborative nella piattaforma UninaTaskBoard. Il sistema dovrà essere collegato a un database relazionale pre-popolato contenente informazioni sugli utenti registrati e sui progetti esistenti. Il sistema deve permettere l'autenticazione degli utenti tramite credenziali. Una volta autenticato, l'utente può creare un nuovo progetto oppure accedere ai progetti ai quali partecipa. Dopo aver selezionato un progetto, l'utente può visualizzare l'elenco delle attività esistenti e crearne di nuove, specificando descrizione, eventuale data di scadenza, tipologia dell'attività e membri responsabili. Deve essere inoltre possibile aggiornare lo stato delle attività e aggiungere commenti. Il sistema deve permettere di filtrare le attività in base allo stato, alla scadenza, al membro assegnato o alla tipologia. Infine, il sistema deve fornire un report che mostri, per ogni progetto, il numero totale di attività, il numero di attività completate, in corso e non iniziate, il numero di attività di sviluppo, il numero medio di revisioni per file di codice e il numero di attività completate da ciascun membro. Il report deve includere una rappresentazione grafica dei dati realizzata con JFreeChart o JavaFX.

1.3 Requisiti

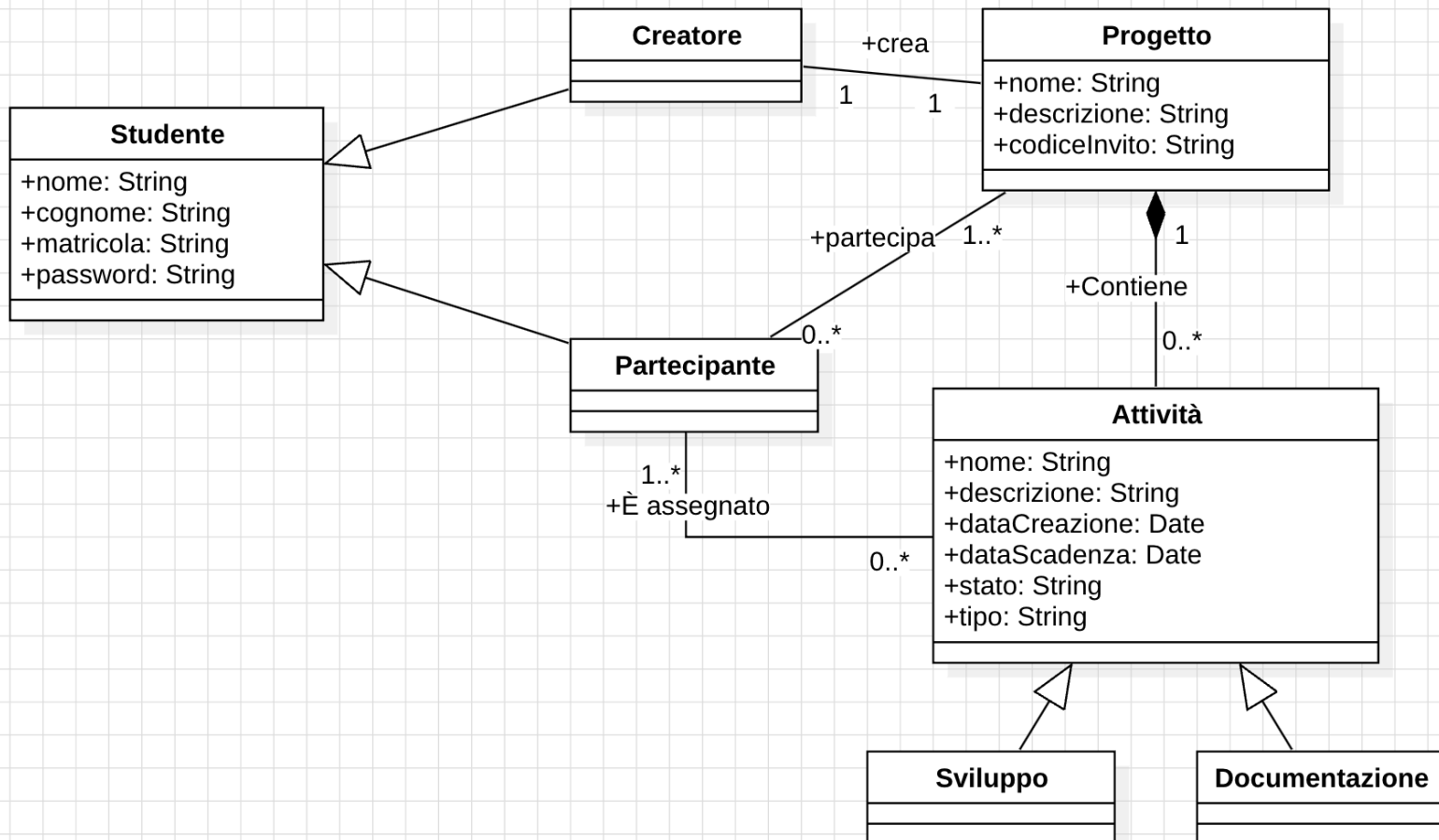
Il sistema deve permettere a uno studente di:

- registrarsi e accedere al sistema;
- creare un nuovo progetto;
- generare un codice invito per il progetto;
- entrare in un progetto tramite codice invito;
- visualizzare i progetti a cui partecipa;

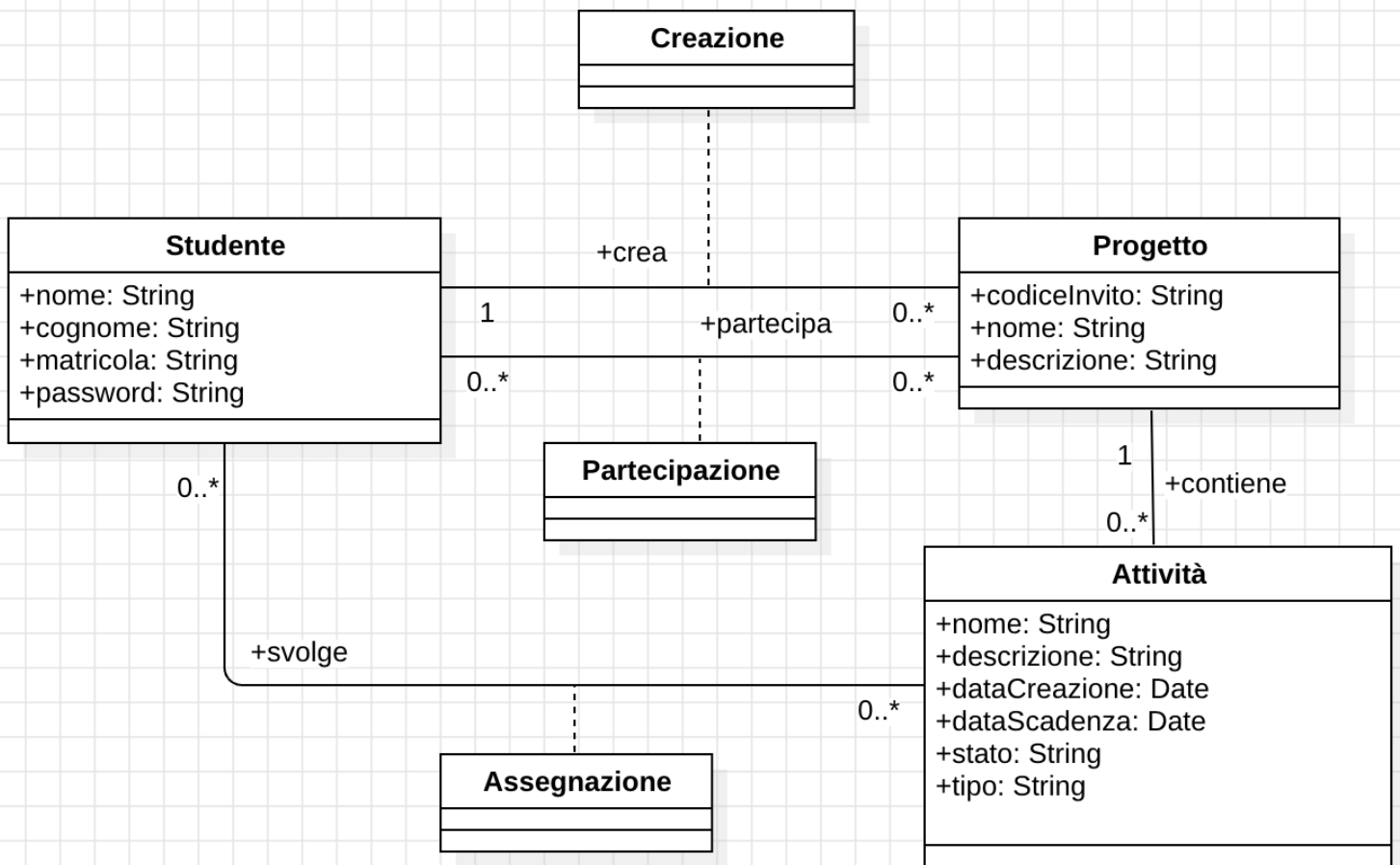
- creare attività all'interno di un progetto;
- assegnare attività a uno o più partecipanti;
- modificare lo stato di un'attività;
- distinguere le attività in attività di sviluppo e attività di documentazione;
- visualizzare le attività di un progetto;
- aggiungere commenti alle attività;
- generare un report del progetto.

Progettazione concettuale

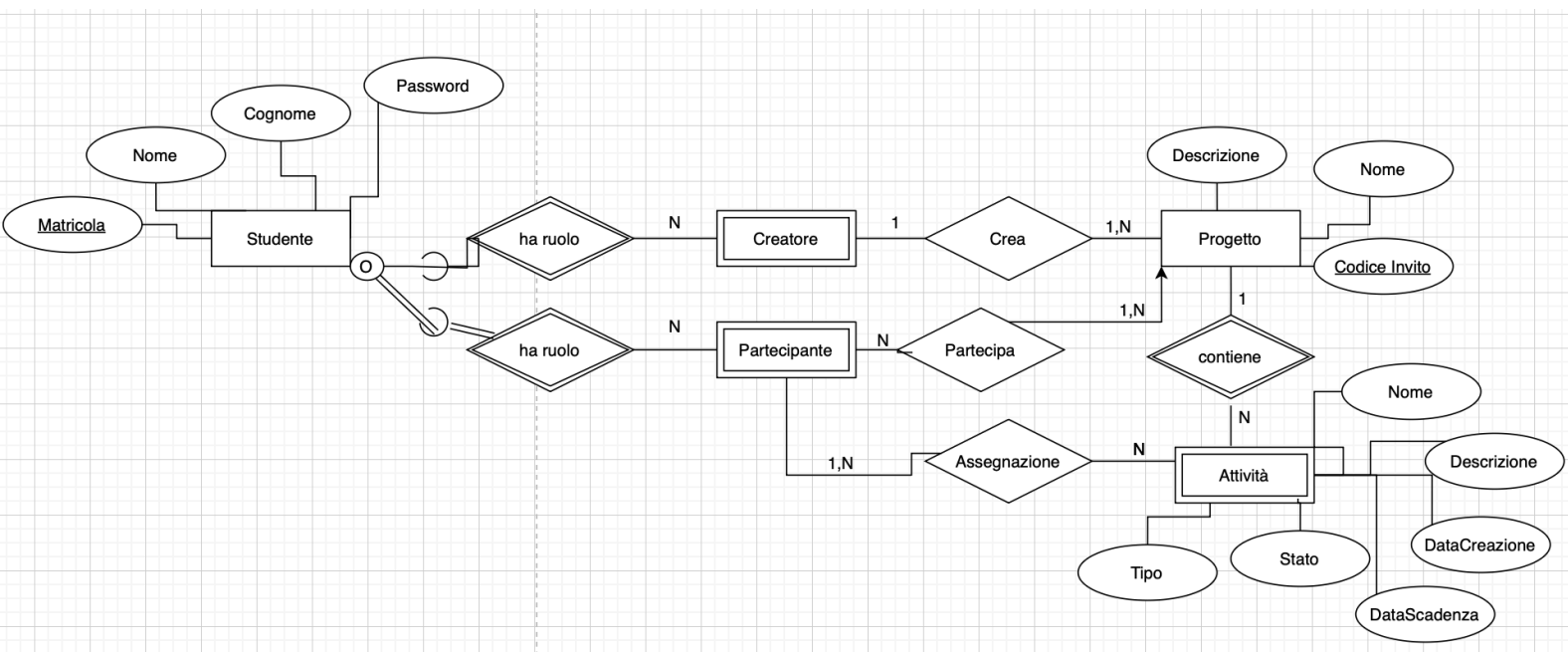
2.1 Class Diagram UML non ristrutturato



2.2 Class Diagram UML Ristrutturato



2.3 Diagramma entità-relazioni (ER)



Dizionario

3.1 Dizionario delle classi

Si danno per scontati attributi banali (ad esempio “nome”)

Dizionario delle classi		
Classe	Descrizione	Attributi
Studente	Rappresenta l'utente che utilizza UninaTraskBoard	Matricola: La matricola dello studente Password: Stringa codificata secondo un algoritmo di sicurezza
Progetto	Struttura di UninaTaskBoard contenente le Attività	Codice Invito: Stringa generata automaticamente al momento della creazione del progetto, univoco del progetto stesso e necessario per aggiungere partecipanti
Attività	Struttura interna al progetto	Stato: Stringa che indica se l'attività è stata completata o meno DataCreazione: Data assegnata automaticamente al momento della creazione dell'attività DataScadenza: La data in cui lo stato passa automaticamente a SCADUTO. Può avere valore nullo Tipo: stringa che indica lo scopo dell'attività
Creazione		
Partecipazione		
Assegnazione		

3.2 Dizionario dei vincoli

Dizionario dei vincoli		
Identificativo	Tipo Vincolo	Descrizione
VI.01 StatoAttività	Dominio	L'attributo stato deve sempre avere una di queste quattro stringhe come valore: - "IN CORSO" - "COMPLETATO" - "SCADUTO" - "NON INIZIATA"
VI.02 TipoAttività	Dominio	L'attributo tipo deve sempre avere una di queste due stringhe come valore: - "SVILUPPO" - "DOCUMENTAZIONE"
VI.03 NomeStudente	Dominio	Il nome di studente non può essere nullo
VI.04 CognomeStudente	Dominio	Il cognome di studente non può essere nullo
VI.05 MatricolaStudente	Chiave primaria	La matricola di uno studente è unico e non nullo
VI.06 IdentificazioneProgetto	Chiave primaria	Il codice invito di un progetto è unico e non nullo
VI.07 PartecipazioneProgetti	Referenziale	Uno studente può partecipare a più progetti e un progetto può avere più partecipanti.
VI.08 AppartenenzaAttivitàProgetto	Referenziale	Ogni attività appartiene a un solo progetto; un progetto può contenere più attività.
VI.09 CoerenzaTemporale	N-PLA	La data di DataScadenza non può essere antecedente la data di DataCreazione
VI.10 PasswordStudente	Dominio	La password di studente non può essere nulla e non può essere più breve di 8 caratteri o più lunga di 18
VI.11 CreatorePartecipante	Referenziale	Il creatore di un progetto ne è automaticamente anche partecipante
VI.12 ScadenzaAttività	N-PLA	Quando la data di scadenza viene raggiunta lo stato di una attività passa automaticamente ad SCADUTO

Schema logico

4.1 Mapping

Lo schema logico, basato sul classi diagram ristrutturato, è quasi privo di chiavi surrogate come identificatori, ad eccezione di Attività, la quale mancava di chiavi candidate adeguate. L'id di Attività verrà generato automaticamente come intero sempre crescente tramite "INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY".

4.2 Schema logico

Studente			
<u>Matricola</u>	Nome	Cognome	Password

Creazione	
<u>MatricolaCreatore</u>	<u>CodiceInvito</u>

Partecipazione	
<u>MatricolaPartecipante</u>	<u>CodiceInvito</u>

Progetto		
<u>CodiceInvito</u>	Nome	Descrizione

Attività							
<u>ID</u>	Nome	Descrizione	DataCreazione	DataScadenza	Stato	Tipo	codiceProgetto

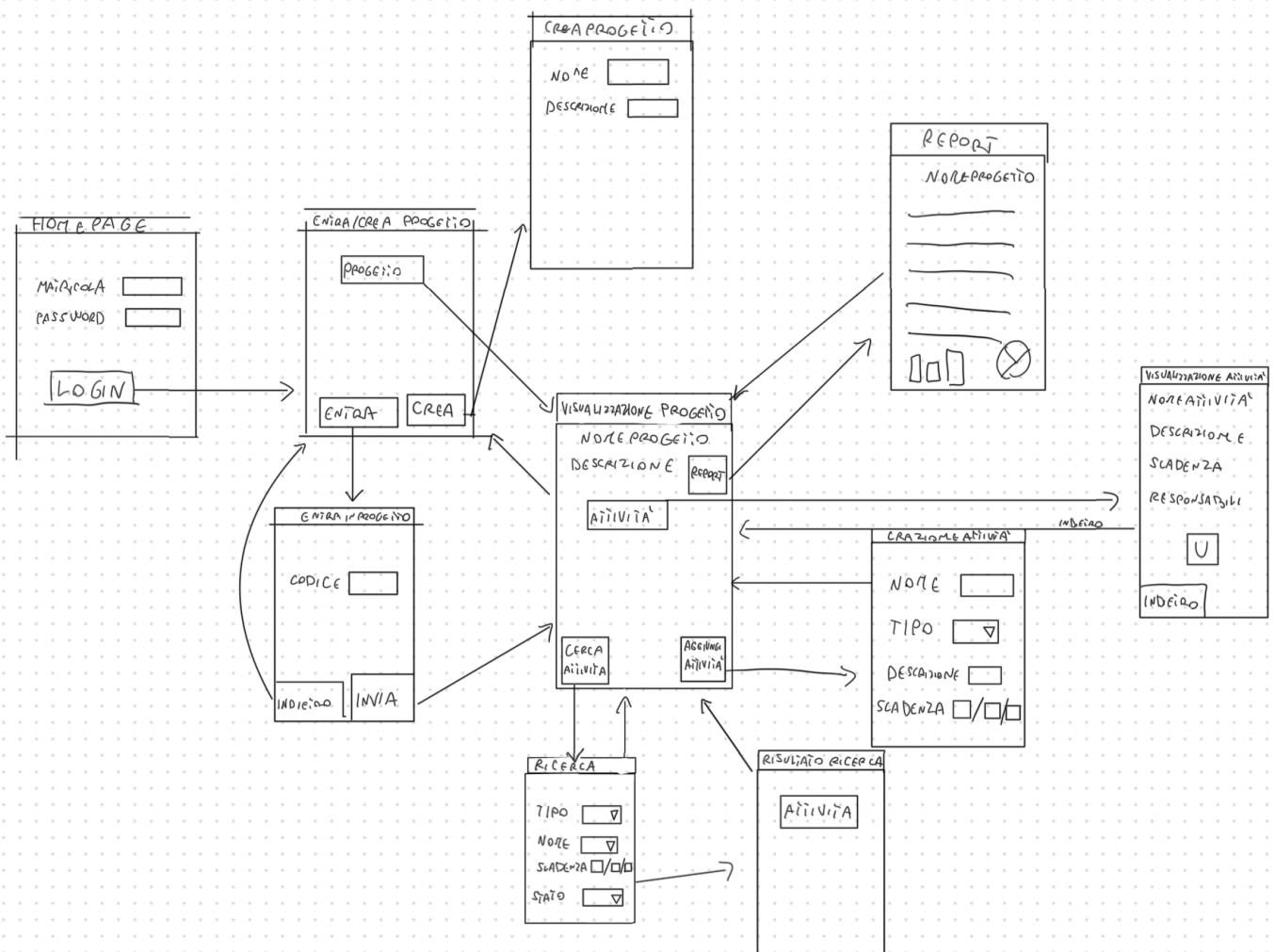
Assegnazione		
<u>MatricolaPartecipante</u>	<u>CodiceInvitoProgetto</u>	<u>IDAttività</u>

Progettazione Object-Oriented

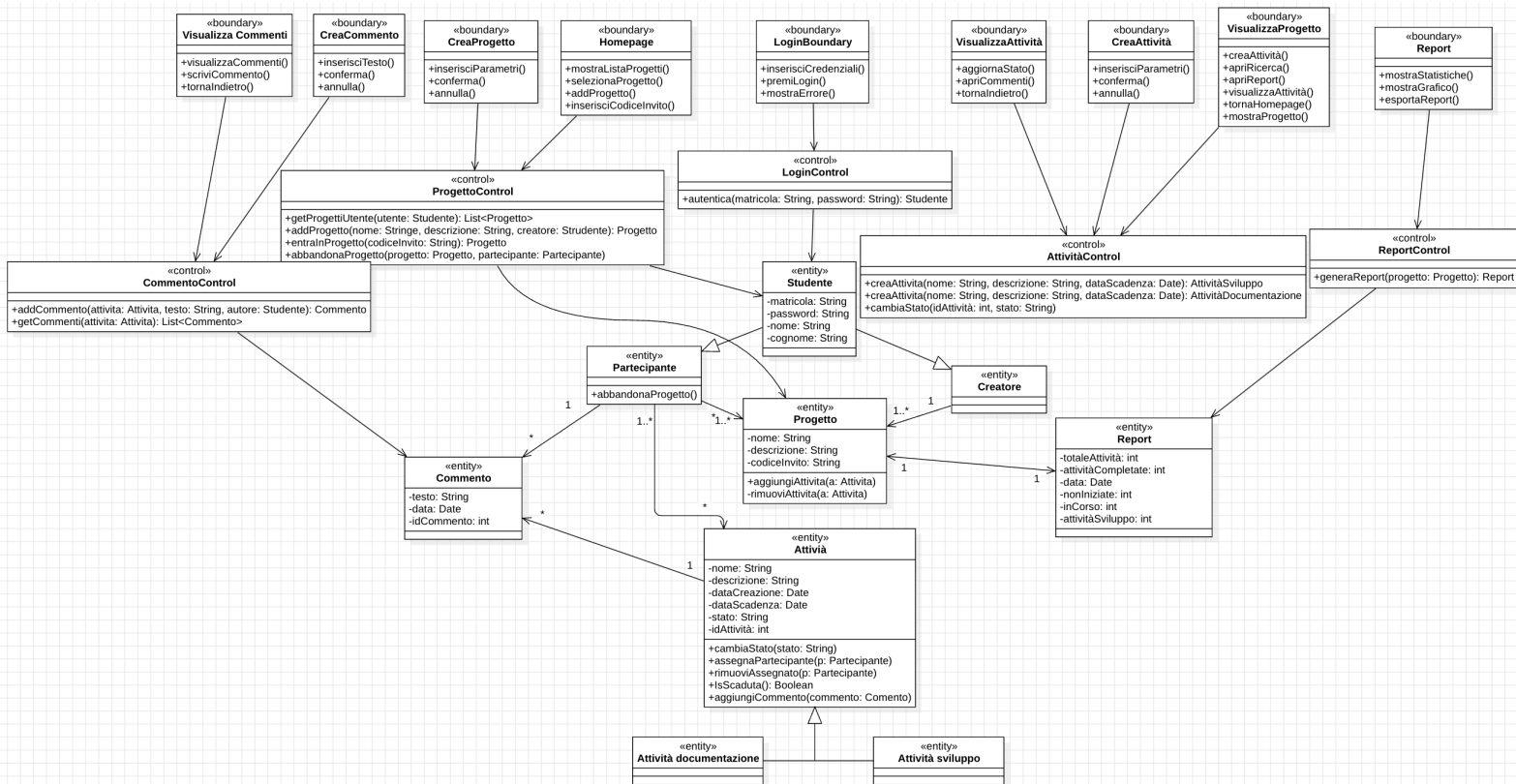
5.1 Mock-up

Il seguente mock-up illustra l'interfaccia dell'applicazione.

Un utente già registrato nel database può accedere tramite matricola e password e partecipare e/o creare un progetto. I progetti contengono molteplici attività, le quali possono essere aggiunte in qualsiasi momento da qualsiasi partecipante. È inoltre in qualsiasi momento possibile visualizzare un report del progetto, il quale illustra molteplici dati utili accompagnato da grafici.



5.2 Class Diagram EBC



Schema fisico

6.1 Tabelle

Di seguito le tabelle scritte in SQL

```
10 CREATE TABLE Studente (  
11     matricola VARCHAR(20) PRIMARY KEY,  
12     nome VARCHAR(50) NOT NULL,  
13     cognome VARCHAR(50) NOT NULL,  
14     password VARCHAR(18) NOT NULL  
15         CHECK (char_length(password) BETWEEN 8 AND 18)  
16 );  
17  
18 CREATE TABLE Progetto (  
19     codiceInvito VARCHAR(20) PRIMARY KEY,  
20     nome VARCHAR(50) NOT NULL,  
21     descrizione VARCHAR(200)  
22 );  
23  
24 CREATE TABLE Creazione (  
25     matricolaCreatore VARCHAR(20) NOT NULL  
26         REFERENCES Studente(matricola),  
27     codiceInvito VARCHAR(20) PRIMARY KEY  
28         REFERENCES Progetto(codiceInvito) ON DELETE CASCADE  
29 );  
30  
31 CREATE TABLE Partecipazione (  
32     matricolaPartecipante VARCHAR(20) NOT NULL  
33         REFERENCES Studente(matricola),  
34     codiceInvito VARCHAR(20) NOT NULL  
35         REFERENCES Progetto(codiceInvito) ON DELETE CASCADE,  
36     PRIMARY KEY (matricolaPartecipante, codiceInvito)  
37 );  
38  
39 CREATE TABLE Attivita (  
40     idAttivita INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
41     nome VARCHAR(100) NOT NULL,  
42     descrizione VARCHAR(200),  
43     dataCreazione DATE NOT NULL DEFAULT CURRENT_DATE,  
44     dataScadenza DATE,  
45     stato VARCHAR(12) NOT NULL DEFAULT 'NON INIZIATO'  
46         CHECK (stato IN ('NON INIZIATO', 'IN CORSO', 'COMPLETATO', 'SCADUTO')),  
47     tipo VARCHAR(15) NOT NULL  
48         CHECK (tipo IN ('SVILUPPO', 'DOCUMENTAZIONE')),  
49     codiceProgetto VARCHAR(20) NOT NULL  
50         REFERENCES Progetto(codiceInvito) ON DELETE CASCADE,  
51     CHECK (dataScadenza IS NULL OR dataScadenza >= dataCreazione)  
52 );  
53  
54 CREATE TABLE Commento (  
55     idCommento INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
56     testo TEXT NOT NULL,  
57     data DATE NOT NULL DEFAULT CURRENT_DATE,  
58     idAttivita INTEGER NOT NULL  
59         REFERENCES Attivita(idAttivita) ON DELETE CASCADE,  
60     matricolaAutore VARCHAR(20) NOT NULL  
61         REFERENCES Studente(matricola)  
62 );
```

```

64 CREATE TABLE Assegnazione (
65     matricolaPartecipante VARCHAR(20) NOT NULL,
66     codiceInvitoProgetto VARCHAR(20) NOT NULL,
67     idAttivita INTEGER NOT NULL
68     REFERENCES Attivita(idAttivita) ON DELETE CASCADE,
69     PRIMARY KEY (matricolaPartecipante, idAttivita),
70     FOREIGN KEY (matricolaPartecipante, codiceInvitoProgetto)
71     REFERENCES Partecipazione(matricolaPartecipante, codiceInvito)
72     ON DELETE CASCADE
73 );
74

```

6.2 Vincoli

I seguenti trigger assicurano il corretto funzionamento del database

```

79 -- Il creatore deve partecipare al suo stesso progetto
80 CREATE OR REPLACE FUNCTION fun_trg_creatore_partecipa()
81 RETURNS TRIGGER
82 LANGUAGE plpgsql
83 AS $$
84 BEGIN
85     INSERT INTO Partecipazione (matricolaPartecipante, codiceInvito)
86     VALUES (NEW.matricolaCreatore, NEW.codiceInvito)
87     ON CONFLICT DO NOTHING;
88     RETURN NEW;
89 END;
90 $$;
91
92 CREATE TRIGGER trg_creazione
93 AFTER INSERT ON Creazione
94 FOR EACH ROW
95 EXECUTE FUNCTION fun_trg_creatore_partecipa();

```

```

98  -- controlla e gestisci lo stato delle attività
99  CREATE OR REPLACE FUNCTION fun_trg_attivita_stato()
100 RETURNS TRIGGER
101 LANGUAGE plpgsql
102 AS $$
103 BEGIN
104     IF NEW.dataScadenza IS NOT NULL
105         AND NEW.dataScadenza < CURRENT_DATE
106         AND NEW.stato <> 'COMPLETATO' THEN
107         NEW.stato := 'SCADUTO';
108     END IF;
109     RETURN NEW;
110 END;
111 $$;
112
113 CREATE TRIGGER trg_attivita_stato
114 BEFORE INSERT OR UPDATE ON Attivita
115 FOR EACH ROW
116 EXECUTE FUNCTION fun_trg_attivita_stato();

```



```

119 -- l'attività assegnata ad uno studente deve appartenere al progetto indicato.
120 CREATE OR REPLACE FUNCTION fun_trg_assegnazione_coerenza()
121 RETURNS TRIGGER
122 LANGUAGE plpgsql
123 AS $$
124 DECLARE
125     codiceProgAttivita VARCHAR(20);
126 BEGIN
127     SELECT codiceProgetto INTO codiceProgAttivita
128     FROM Attivita
129     WHERE idAttivita = NEW.idAttivita;
130
131     IF codiceProgAttivita IS DISTINCT FROM NEW.codiceInvitoProgetto THEN
132         RAISE EXCEPTION
133             'L'attività % non appartiene al progetto %',
134             NEW.idAttivita, NEW.codiceInvitoProgetto;
135     END IF;
136     RETURN NEW;
137 END;
138 $$;
139
140 CREATE TRIGGER trg_assegnazione_coerenza
141 BEFORE INSERT OR UPDATE ON Assegnazione
142 FOR EACH ROW
143 EXECUTE FUNCTION fun_trg_assegnazione_coerenza();

```